Let's start with the setup. By default, RSpec is configured to look in `spec/fixtures` for fixtures. This is correct for a project using RSpec from the start, but for us, the fixtures are in `test/fixtures`. Change this by editing `spec/rails_helper.rb`:

rails80/depot_xa/spec/rails_helper.rb

```
    RSpec.configure do |config|
      # Remove this line if you're not using ActiveRecord or ActiveRecord
    fixtures
      config.fixture_paths = [
»       Rails.root.join('test/fixtures')
      ]
```

Back to the spec—its setup will need to create a `Cart` to use in our tests as well as fetch two products from fixtures. By default, fixtures aren't available in specs, but you can call `fixtures` to make them available. Here's what the setup looks like:

rails80/depot_xa/spec/models/cart_spec.rb

```
    require 'rails_helper'

    RSpec.describe Cart, type: :model do

»     fixtures :products
»     subject(:cart) { Cart.new }
»
»     let(:book_one) { products(:pragprog) }
»     let(:book_two) { products(:two) }
```

This definitely doesn't look like our original test! The call to `subject` declares the variable `cart`, which you'll use in the tests later. The calls to `let` declare other variables that can be used in the tests. The reason for two methods that seemingly do the same thing is an RSpec convention. The object that's the focus of the test is declared with `subject`. Ancillary data needed for the test is declared with `let`.